

Exercise Collection 2xHIF

Exercises from the collection
created by DI Dr. Heinz Schiffermüller

Edited and supplemented by DI Johannes Loibner, BSc

October 2024

Exa_107_NumberAnalysis

Create the class `NumberAnalysis` according to the class diagram.

Additionally, create constructor(s), as well as getter- and setter methods as needed.

The method `calcEvenOdd()` is used to find out, if the number is even or odd. `calcPrime()` is used to find out, if the number is a prime number. Within `calcDigitSum()` the sum of all digits of the number is calculated and saved. Finally, the `calculate()` method must be used to analyze the number.

With the help of the `toString()` method, the result of the analysis is returned as a `String`. Please take a look at the demo output below.

NumberAnalysis
- number : int - even : boolean - prime : boolean - digitSum : int
+ calculate() : void + toString() : String - calcEvenOdd() : void - calcPrime() : void - calcDigitSum() : void

The program should ask the user for more and more numbers, until a 0 is entered.

Class, Object, Attribute, Method

Demo Output:

Welcome to number analysis!

Please enter a number: 12

12 is even, not prime, digit sum: 3

Please enter a number: 9

9 is odd, not prime, digit sum: 9

Please enter a number: 0

Bye! :)

Exa_201_LotteryNumbers

Create the class `LottoNumbers` according to the class diagram.

Additionally, create constructor(s), as well as getter- and setter methods as needed.

Create 6 different random numbers in the interval [1;45] inside the constructor. Additionally, another number, also different from all the numbers already generated, should be created and written to `additionalNumber`.

Within the `sortNumbers()` method, the array is being sorted in an ascending order. You should use the basic version of Bubble Sort.

The `toString()` method returns the output `String` as shown in the demo output.

LottoNumbers
- numbers : int[] - additionalNumber : int
+ sortNumbers() : void + toString() : String

Constructor

Demo Output:

Lottery Result: 2, 9, 12, 13, 30, 44 Additional Number: 11

Exa_202_Fibonacci

Create the classes according to the class diagram.
Additionally, create constructor(s), as well as getter- and setter methods as needed.

The fibonacci series is created by calculating the sum of the two preceding numbers: 1,1,2,3,5,8,13,21,34,...

Read a valid whole number (upperLimit) from the console and calculate the sum of all fibonacci numbers that are even and smaller or equal to the limit.

Use the constructor to initialize all the instance variables with the help of the methods. computeSum() calculates the sum as defined above. The isEven(int number) method returns true if the passed number is even, false otherwise.

Finally, toString() is used to return the output String is shown in the demo output.

Fibonacci
- upperLimit : int - fiboSum : int
- computeSum() : void - isEven(number : int) : boolean + toString() : String

FibonacciUI
+ main(args : String[]) : void

Constructor, Classes

Demo Output:

Upper limit: 4000000
Fibonacci sum til 4000000: 4613732

Exa_203_Triangle

Create the classes according to the class diagram.
Additionally, create constructor(s), as well as getter- and setter methods as needed.

Read in the 3 sides of a triangle and use the constructor to initialize them. Calling the sortSides() method sorts them in an ascending order so that the outcome is $a \leq b \leq c$.

With the determineType() method, the type of the triangle is defined:

- equilateral ($a = b = c$)
- isosceles ($a \neq b = c$, $a = b \neq c$, $a = c \neq b$)
- right-angled ($a^2 + b^2 = c^2$)
- general ($a + b > c$)
- not a triangle

Triangle
- sideA : int - sideB : int - sideC : int - type : String
+ sortSides() : void + determineType() : void + toString() : String

TriangleUI
+ readSide(side : String, scan : Scanner) : int + main(args : String[]) : void

The `toString()` method is used to return the output String.

The method `readSide()` in the `TriangleUI` class is used to read always **one** side and return the value.

Extensions:

- Create a non-parameter constructor and create random values for the three sides between 5 and 10. Test this in your `main()` method.

Constructor, Classes

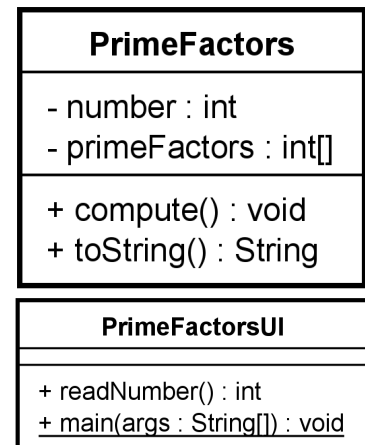
Demo Output:

```
Enter side A: 5
Enter side B: 3
Enter side C: 4
Triangle (3,4,5) -> right-angled
```

Exa_204_PrimeFactors

Create the classes according to the class diagram.
Additionally, create constructor(s), as well as getter- and setter methods as needed.

Read a number from the console and use a setter method to initialize the according instance variable. The `compute()` method calculates all the prime factors and saves them in the array. At the end, the array is not allowed to be bigger than the number of prime factors. You are not allowed to use any other data structures. Try to use ChatGPT to find a solution for the array problem.
The `toString()` method returns the output as stated in the demo.



The method `readNumbers` shall ask the user for new numbers, until a number > 0 is entered.

Constructor, Setter, Classes

Demo Output:

```
Number: 13195
Prime factors of 13195: 5, 7, 13, 29
```

Exa_205_NumberSorting

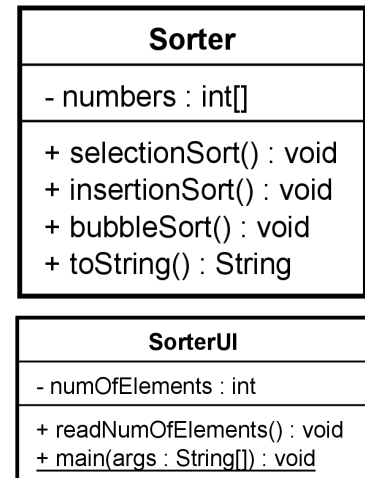
Create the classes according to the class diagram.

The numbers are saved in an integer array named `numbers`. The size of the array is entered by the user and passed to the constructor of the class `Sorter`. Within the constructor the array is initialized with random values $0 < \text{rn} < \text{sizeofArray}$.

Using the menu in the console, the user can select the sorting algorithm.

In the output, the user only sees the first 10 and the last 10 numbers of the array. If the array has less than 20 elements, all the elements are shown. Stick to the demo output!

Extensions:



- Create a new method `quickSort()` in the `Sorter` class to implement this recursive algorithm.
- Change the menu in a way, that the user can always start from the beginning. For every new start, a new object of the class `Sorter` needs to be created.
- Create a time tracking in the `main()` method to measure how long the sorting takes and print it to console at the end of sorting. (You may ask ChatGPT for help!)

The `readNumOfElements()` method is used to read the number of integers to be created. You might need a getter method here.

Constructor, Classes

Demo Output:

```
Number of elements: 100
(1) Selection Sort
(2) Insertion Sort
(3) Bubble Sort
Please select the algorithm: 2
2, 3, 3, 5, 6, 6, 9, 10, 10, 14
...
84, 84, 88, 88, 92, 93, 94, 97, 97, 98
```

CodingBat - String

CodingBat is a platform to learn Java programming with simple and small exercises. You are asked to go to codingbat.com/java and register there. Go to prefs on the top right corner and enter the email address of your teacher to share the results. **This is not optional.**

Now do all the exercises within the `String-1` task collection.

String

Exa_206_IQ

Exa_207_AddressManagement

Create the classes according to the class diagram. The aim of this programm is to read and change an address.

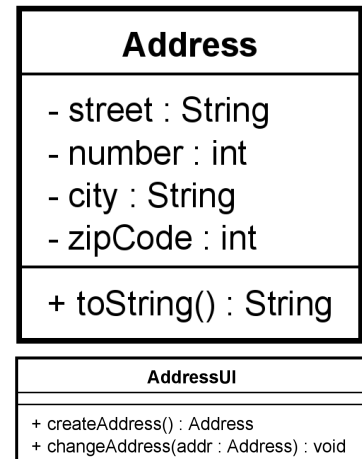
The class Address is a data class with a constructor, getter- and setter methods, and a toString() method.

Within the class AddressUI, a new object of Address is created (createAddress()) and can also be changed (changeAddress()). Please stick to the defined parameters and return types.

2-classes, constructor, getter-setter methods

Demo Output:

```
Enter street: Rodeo Drive
Enter number: 123
Enter city: LA
Enter zipcode: 90210
Address: Rodeo Drive 123 - 90210 LA
What would you like to change?
(A) street: Rodeo Drive
(B) number: 123
(C) city: LA
(D) zipcode: 90210
a
Enter new value: Sunset Blvd
Address: Sunset Blvd 123 - 90210 LA
```



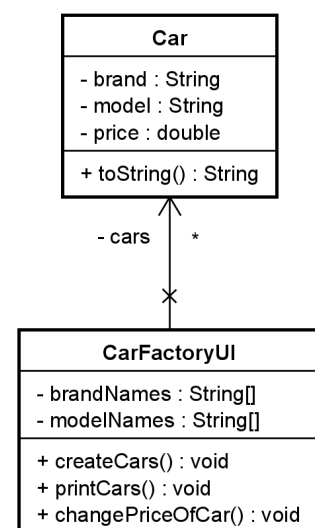
Exa_208_CarFactory

Create the classes Car and CarFactoryUI according to the class diagram. The aim of the program is to create and change car data. The Car class is a data class and contains a constructor, getter- and setter methods, and a toString() method.

Within the CarFactoryUI class, there is a String-array holding at least 5 different car brands (brandNames), a String-array holding at least 10 different car models (modelNameNames), and an array holding Car objects.

In the main method, the number of cars should be read from the console.

The method createCars() creates as many random Car objects as the user defined before and saves them into the cars array. The brand and the model are chosen randomly from the corresponding arrays and the price is also a random value from the interval [1000;10000[.



With the help of `printCars()` all cars in the cars array are printed as shown in the demo output.
The method `changePriceOfCar()` changes the price of a selected car.
Please stick to the demo output.

2-classes, constructor, getter-setter methods

Demo Output:

```
Enter number of cars: 4
1      Audi-Panda costs € 5708,96
2      Fiat-500 costs € 9261,23
3      VW-Polo costs € 1456,47
4      Audi-Golf costs € 1781,31
Change price of car: 2
Car: Fiat-500
Actual price: 9261,23 €
New price: 6666
1      Audi-Panda costs € 5708,96
2      Fiat-500 costs € 6666,00
3      VW-Polo costs € 1456,47
4      Audi-Golf costs € 1781,31
```

Exa_209_TrainingsCalculation

Create the classes according to the class diagram. The aim of this program is to create a calculation trainer with different math tasks in two different difficulty levels.

The class `TrainingsCalculation` holds a calculation. Within the constructor, all the instance variables need to be initialized. Within difficulty level 1, random numbers in the interval `[1;10]` need to be created. In difficult level 2 it's random numbers in the interval `[1;100]`. The operator is randomly chosen from the following: `+`, `-`, `/`, `*`. The result is always a whole number.

Within the `toString()` the calculation (without the result) is returned as shown in the demo output.

CalcTrainerUI
+ performTrainingsTask(difficulty : byte, round : int) : boolean

TrainingsCalculation
- number1 : int
- number2 : int
- operator : char
- result : int
- difficulty : byte
+ toString() : String

The `performTrainingsTask()` method in the `CalcTrainerUI` class is responsible to output and perform a single calculation task. For each round:

- the task needs to be printed
- the result should be entered by the user
- a feedback is printed according to the demo output

At the end, the number of achieved points is shown.

2-classes, constructor, getter-setter methods

Demo Output:

```

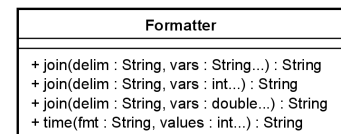
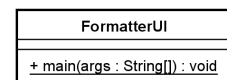
Enter number of rounds: 3
Enter difficulty: 1
Round 1
    1 * 10 = ?
    Enter result: 10
    perfect
Round 2
    8 - 10 = ?
    Enter result: 5
    The correct result is -2
Round 3
    1 - 1 = ?
    Enter result: 0
    perfect
You achieved 2 of 3 points

```

Exa_210_Formatter

Create the classes according to the class diagram.

The `join()` method is used to concatenate Numbers or Strings with a defined Delimiter. The values are passed to the method and the String is returned. Make sure that double-values are formatted with 2 decimal places only.



The `time` method is used to format a time, defined by hours, minutes, and seconds - in exactly that order. The following combination are valid: hours; hours, minutes; hours, minutes, seconds. The format `fmt` defines which String representation of the time needs to be returned. The following formats are supported:

- `hh:mm:ss` -> 14:20:30
- `HH:mm:ss` -> 02:20:30 PM
- `hh:mm` -> 07:20
- `HH:mm` -> 07:20 AM

For a wrong number of parameters or an invalid format, a empty String is returned.

Within the class `FormatterUI` it is important to test all cases (without any user input). Make sure to have at least 10 test cases. Here are examples for some test cases:

- Monday-Tuesday-Wednesday-Thursday-Friday
- 1,2,3,4
- 1
- 0,50 - 0,33 - 0,25
- 12:20:30

- 10:45
- 06:25 PM
- 11:33:00 AM

When done, the following extensions can be implemented:

- Add 3 further overloads of the `join()` method with the same functionality, but with a `char` delimiter.
- Create a method `date` to support different date formats.

var args, overloading methods

Exa_211_GradeCalculation

Create the classes according to the class diagram. Within the program, grade should be evaluated.

Within the constructor of the class `GradeBL`, the array `grades` is initialized with random grades in the interval `[1;5]`. The size of the array is entered by the user. The array `gradeNames` is literally initialized with the German names for the grades. The `gradeStatistics` array should hold the frequency for each grade.

The method `evaluation()` is responsible to find frequency and average of the grades. The `toString()` method prints all the grades as shown in the demo output.

The aim of the class `GradeUI` is to create the defined demo output.

2-classes, constructor, getter-setter methods

GradeBL
- grades : int[] - gradeStatistics : int[] - gradeNames : String[] - avgValue : float
+ evaluation() : void + toString() : String

GradeUI
+ main(args : String[]) : void

Demo Output:

Enter the number of grades: 36

4, 2, 4, 2, 1, 3

4, 4, 4, 4, 5, 2

5, 1, 4, 4, 4, 4

2, 5, 5, 2, 3, 3

1, 3, 1, 4, 3, 3

4, 5, 1, 1, 3, 4

Evaluation:

Sehr Gut: 6

Gut: 5

Befriedigend: 7

Genügend: 13

Nicht Genügend: 5

Average: 3,17

Exa_301_MyMath

Please implement the **constant** PI to hold the number with 15 decimal places. Additionally, implement the following static methods:

- `abs(int a)`: Returns the absolute value of a.
- `min(int a, int b, int c)`: Returns the minimum value of the three passed
- `max(int a, int b, int c)`: Returns the maximum value of the three passed
- `power(double basis, int exp)`: Returns the basis to the power of exp as a double value. It should also work for negative basis and exponents.
- `round(double value, int positions)`: Returns the value rounded with the amount of decimal places (`positions`).
- `sin(double x)`: Returns the sin of x.
- `cos(double x)`: Returns the cos of x.

MyMath
+ PI : double
+ abs(a : int) : int
+ min(a : int, b : int, c : int) : int
+ max(a : int, b : int, c : int) : int
+ power(basis : double, exp : int) : double
+ round(value : double, positions : int) : double
+ faculty(number : int) : long
+ sin(x : double) : double
+ cos(x : double) : double

MathTesterUI
+ main(args : String[]) : void

For all calculations your are NOT allowed to use the API class Math. Please use the following functions to calculate the sin and cos by using the first **10** terms only. Use your already implemented methods.

$$\sin(z) = \sum_{k=0}^{\infty} \frac{(-1)^k \cdot z^{2k+1}}{(2k+1)!} = z - \frac{z^3}{3!} + \frac{z^5}{5!} - \frac{z^7}{7!} \pm \dots$$

$$\cos(z) = \sum_{k=0}^{\infty} \frac{(-1)^k \cdot z^{2k}}{(2k)!} = 1 - \frac{z^2}{2!} + \frac{z^4}{4!} - \frac{z^6}{6!} \pm \dots$$

Within the class MathTesterUI make sure to test all methods (without user input). Make sure to test at least the following test cases:

- Test the absolute (`abs`) value with negative inputs
- Test the `max` and `min` methods
- Print the number π
- Round π to 6 decimal places: 3.1415930000000
- Test the faculty method
- Test the following input values for `sin` and `cos`: 0, $\frac{\pi}{4}$, $\frac{\pi}{2}$, π

Make sure to compare the results with correct and good sources.

2-classes, static elements, constants

Exa_401_CurrencyConverter

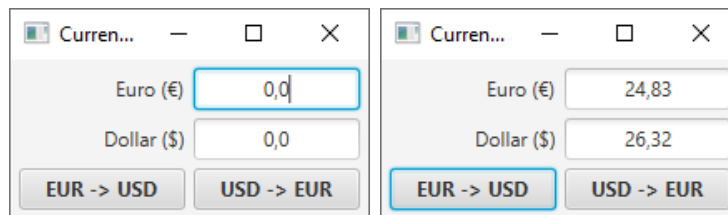
Create a new project and stick to the defined JavaFX setup. In the `pojos` package create the class `CurrencyConverter` and implement both the methods which convert a value from euro to dollar or vice versa.

Please create the GUI according to the demo shown below.

CurrencyConverter
+ <code>euroToDollar(euro : double) : double</code>
+ <code>dollarToEuro(dollar : double) : double</code>

GUI, static elements

Demo Output:



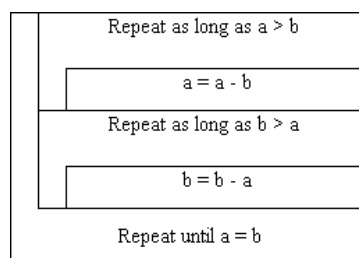
Exa_402_Fraction

Create a new project and stick to the defined JavaFX setup. In the `pojos` package create the class `Fraction` with an constructor to initialize the instance variables. The method `shorten` is used to find the greatest common divisor (GGT) of both numbers and to shorten the fraction.

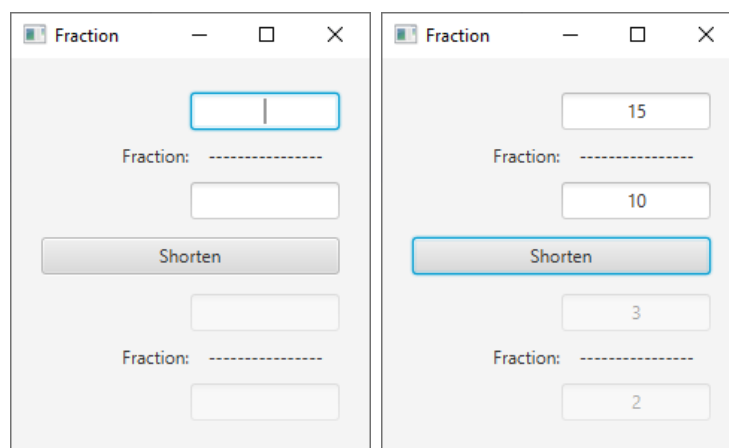
The greatest common divisor of two numbers (a , b) is calculated with the algorithm shown below. Please create the GUI according to the demo shown below.

Fraction
- numerator : int - denominator : int
+ shorten() : void

GUI, classes, constructor, algorithm



Demo Output:



Exa_403_MyDateTimeFormatter

Create a new project and stick to the defined JavaFX setup. In the `pojos` package create the class `MyDateTimeFormatter` according to the class diagram. The aim of the program is to format a date and/or a time. The following table shows the different formatting options.

MyDateTimeFormatter
+ format(formatStr : String, values : int...) : String

Format	String	Output
Date	D	04.10.2024
Date & Time	DT	04.10.2024 - 07:30
Time	T	07:30

Which of the options is used, is decided with the checkboxes. Make sure to handle all Exceptions and use `Alert` to inform the user about errors. Only check the text field inputs which are needed for the selected format.

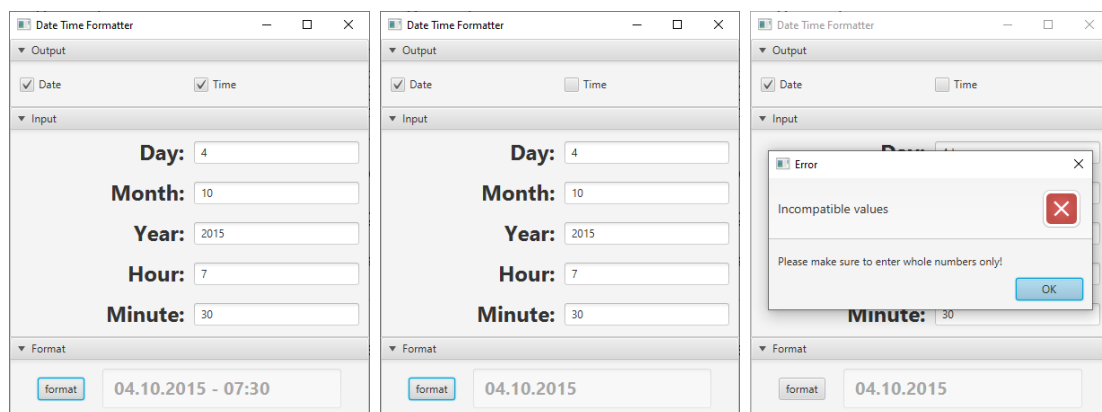
The method `format()` returns a `String` containing the formatted values. The following rules are important:

- All values (except the year) must be represented with 2 digits.
- The number of parameters need to match the number of needed values: D(3), T(2), DT(5)
- Throw a `NumberFormatException` if the number of parameters is not correct.

In the `AppController` create the method `onFormat()` to handle the button click, to format the inputs and print the output.

GUI, Strings, Var-args, static

Demo Output:

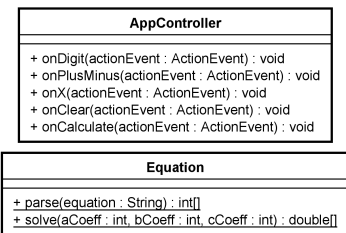


Exa_404_EquationSolver

Create a new project and stick to the defined JavaFX setup. In the `pojos` package create the class `Equation` according to the class diagram. The aim of the program is to solve quadratic equations of the form $ax^2 + bx + c = 0$ with the help of the formula $x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$.

The input of the equation has to follow the format $ax^2 + bx + c = 0$ whereas `a`, `b`, `c` are any integer numbers. The program should only solve equations with all of the three parts. If `a` or `b` are equal to 1, the value can be omitted: e.g.: $x^2 + x - 2 = 0$.

The following is important about the `AppController`:



- `onDigit()`: Method to display one of the 10 digits (0,1,...,9). Leading zeros are not allowed!
- `onPlusMinus()`: Method to display the plus and minus sign. Make sure that they can only be placed at valid positions.
- `onX()`: Method to display `x` or x^2 . Make sure that only the correct order is possible and both `x` and x^2 can only be there once.
- `onCalculate()`: This method is called when the `=0` button is pressed. It displays the `=0` on the correct position only. If so, the equation should be correct and the calculation can be done. Display both solutions (`x1`, `x2`) or `Error`.
- `onClear()`: Method to clear the display.

The following is important about the `Equation` class:

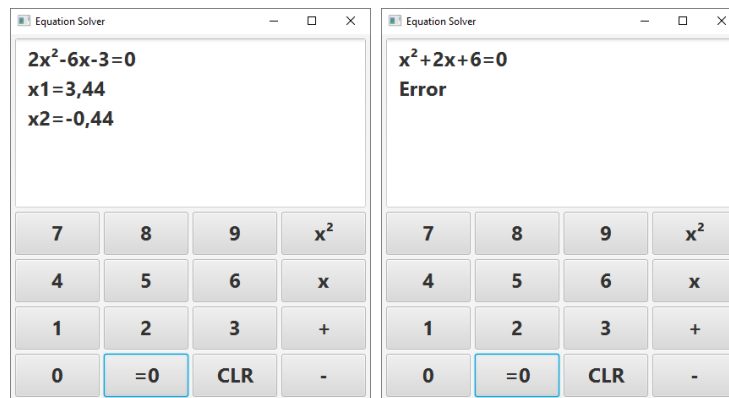
- `parse()`: Implement this method according to the class diagram. This method extracts the values for `a`, `b`, `c` and returns them in an `int` array.
- `solve()`: Implement this method according to the class diagram. This method solves the equation with the formula shown above. Return an `Exception` if the value in the square root function is negative. Otherwise return both of the solutions inside an array.

Make sure that the following test cases are successful:

- $2x^2 - 6x - 3 = 0$
- $x^2 + x - 1 = 0$
- $-x^2 + 12x - 2 = 0$

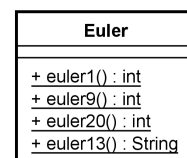
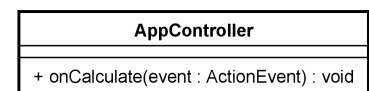
GUI, static

Demo Output:



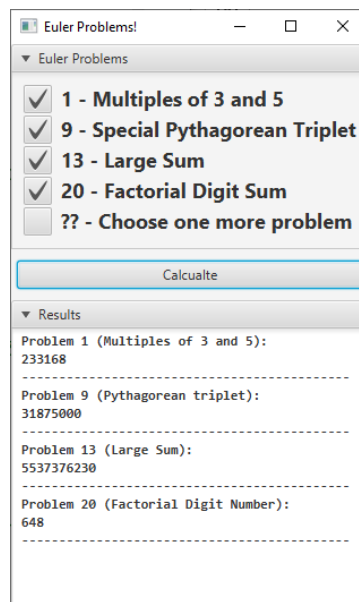
Exa_405_EulerProblems

Create a new project and stick to the defined JavaFX setup. In the pojoes package create the class Euler according to the class diagram. The aim of the program is to solve Euler's problems 1, 9, 13, 20. Additionally, select one other Euler problem and create another method in the Euler class to solve it. Here you can find a documentation of all the Euler problems: <https://projecteuler.net/archives>



GUI, static, algorithms

Demo Output:



Exa_406_ZodiacSign

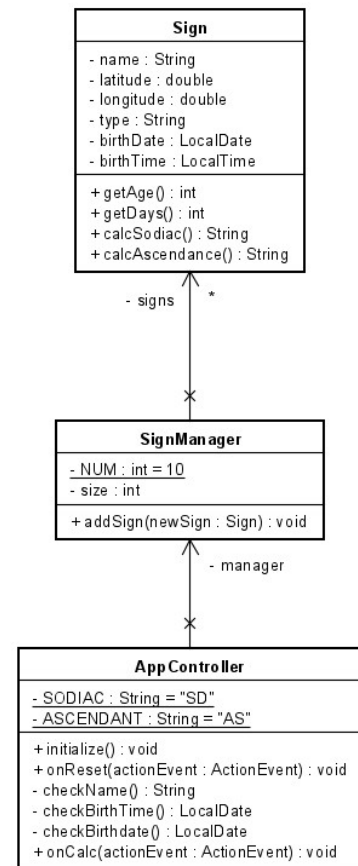
Create a new project and stick to the defined JavaFX setup. The `pojos` package contains the class `Sign` as shown in the class Diagram. The class offers two constructors: one with all values necessary to calculate the zodiac sign, and one with all values necessary to calculate the ascendance.

The method `getAge()` returns the age in years whereas `getDays()` returns the age in days. The methods `calcZodiac()` and `calcAscendance()` return the calculated signs as `String`.

The class `SignManager` is part of the controller package is responsible to hold all signs which are correctly entered by the user. If the maximum amount of 10 signs is reached, an `ArrayIndexOutOfBoundsException` is thrown.

Finally, the `AppController` defines some additional attributes. The `initialize()` method is used to initialize the `SignManager`. The methods `checkName()`, `checkBirthdate()`, and `checkBirthTime()` are used to check the user inputs. If the inputs are not correct, `Exceptions` are thrown. They need to be caught in the `onCalc()` method and messages need to be shown to the user.

The `onReset()` method is used to clear all fields!



If the user wants to calculate more than **10** signs, an error needs to be shown to the user. Make sure that no input errors are affecting the program!!

GUI, static, algorithms, Exceptions

Demo Output:

The screenshot shows a JavaFX window titled "Zodiac Sign". It contains an "Input" section with fields for Name (Johannes), Birthdate (24.07.1993), Birthtime (12.45), Birthplace latitude (45.22344223), and Birthplace longitude (15.66454344). Below these fields are two radio buttons: "Zodiac Sign" (selected) and "Ascendant". At the bottom of the input section are "CALC" and "RESET" buttons. The "Output" section displays the result: "Your zodiac sign is LION!" in bold, followed by "Johannes, you are 31 years old." and "This is 11499 days!" in red text.

Exa_407_MusicApp

Create a music app to add different songs with their interpret, length of the song, and the release date. All the songs are sorted by their length and are saved in an array. Using the arrows, someone can view the different songs saved.

`pojos.Interpret:`

- This class acts as the interpret for a song

`pojos.Song:`

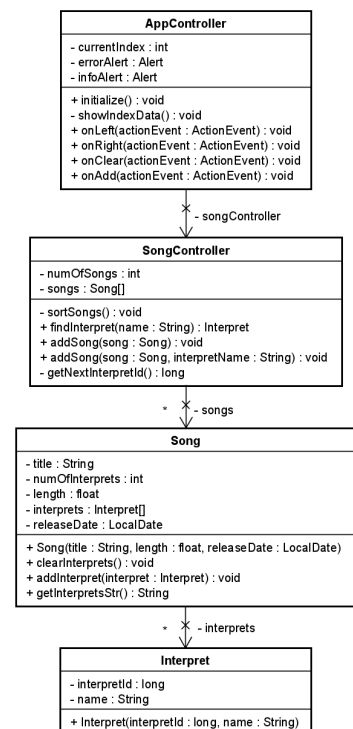
- This class represents a song
- A maximum of four interprets per song is allowed
- `clearInterprets()` just removes all interprets of that song
- `addInterpret()` adds a new interpret to the song
- `getInterpretStr()` returns all interpret names, separated by a pipe (|)

`controller.SongController:`

- This class manages all the songs of the application in an array
- `sortSongs()` sorts all songs by their song length
- The songs need to be sorted at all times
- `findInterpret()` is used to find an already existing interpret and return it; otherwise return false
- `addSong(Song song)` inserts a new song to the array; this method is called if there is no interpret present
- `addSong(Song song, String interpretName)` is used to insert a new song which also contains interprets; make sure to clear any existing interprets of the passed song first, by calling `clearInterprets()`
- Throw a `TooManyInterpretsException` if the there are more than the maximum of interprets
- If a new interpret needs to be created as it is not present yet, use `getNextInterpretId` to find the next highest id to be assigned to the new interpret

`controller.AppController:`

- This class holds an object of the `SongController`
- In `initialize()` all instance variables are initialized

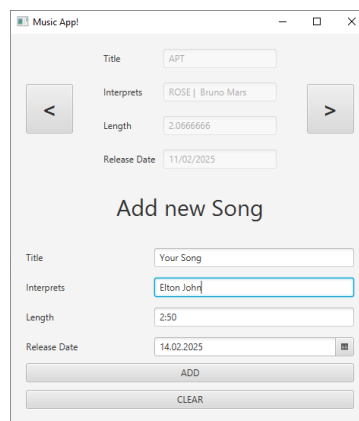


- `currentIndex` always is the index of the song element to be currently visible in the GUI
- `showIndexData()` is responsible to display all song data of the current index in the GUI
- `onLeft()` shows one element to the left by changing `currentIndex` if possible
- `onRight()` shows one element to the right by changing `currentIndex` if possible
- `onClear()` clears out all the input fields
- `onAdd()` adds a new song and checks if all data is entered and valid

Make sure that no errors occur and inform the user if there are errors he needs to take care of. You can find a demo video here.

GUI, Exceptions, LocalDate, Varargs

Demo Output:

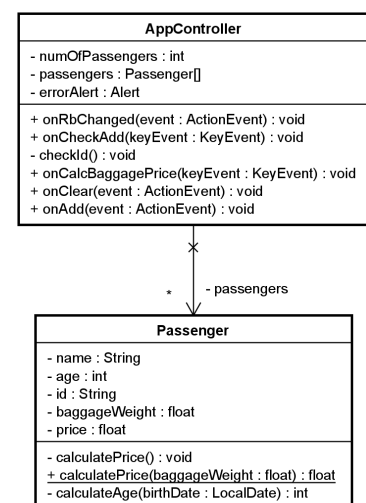


Exa_408_AirportManagement

Create an airport management to manage passengers. The employee can add the name of the passenger, the birth date, some verification (passport or id card) and the baggage weight. The baggage weight can also be empty. 30 kg are free of charge, for overweight a fee needs to be paid. The passengers are saved to an array and shown in the text area below. The ADD button is only enabled if all needed fields are filled and if the verification is valid. Otherwise the button is disabled.

`pojos.Passenger:`

- This class represents a passenger with `name`, `age`, `id`, `baggageWeight`, `price`
- The constructor takes the first and last name, the birth date (`LocalDate`), the `id`, and the baggage weight



- The name of the passenger is created so that the following format is saved: `lastname, firstname`
- Additionally, the age and the price need to be calculated with the creation of a new passenger object
- `calculatePrice()` is used to calculate the baggage price depending on the weight and save it to the instance variable. Use the static method for this.
- `calculatePrice(float baggageWeight)` calculates and returns the price for the passed weight:
 - Up to 30 kg there is no fee charged
 - The next 5 kg cost 5 € per kg
 - The next 5 kg cost 7 € per kg
 - The next 5 kg cost 9 € per kg, ...
 - e.g.: 30 kg = 0 €, 31 kg = 5 €, 35 kg = 25 €, 36 kg = 32 €, ...
- `calculateAge` calculates and sets the age (in years) of the passenger
- The `toString()` method returns the passengers as a `String` as shown in the text area in the demo output

`controller.AppController:`

- This class manages all the passengers of the application in an array
- All instance variables are initialized in the `initialize()` method
- Make sure, that the ADD button is only visible, if all necessary fields are filled out and the verification is valid
- `onRbChanged()` is used as an `onAction` event on the radio buttons to change the text of the according label
- `onCheckAdd()` is called on each necessary text field as a `onKeyTyped` event to activate or deactivate the ADD button (only the presence of content is checked here)
- `checkId()` checks the passport or id card number and activates or deactivates the ADD button
- These are the rules for a passport number to be valid
 - Exactly 7 digits
 - Must start with an upper case letter
 - Only numbers for the rest of the digits
- These are the rules for an id card number to be valid
 - Exactly 12 digits
 - Must start with an upper case letter
 - Followed by 7 numbers, one digit as the check sum and 3 more numbers

– This is how the digit sum is calculated

- * Convert the letter to a number (A = 10, Z = 35)
 - * Then calculate the sum of the multiplications of each of the first 8 digits with 2, 3, 4, ...
 - * Then call the following formula $11 - (sum \% 11)$
 - * If the result is 10 or 11, the check sum is 0
 - * e.g: J04891540877 = $19*2 + 0*3 + 4*4 + 8*5 + 9*6 + 1*7 + 5*8 + 4*9 = 231 \% 11 = 0$; $11 - 0 = 11$; checksum = 0
 - * Valid numbers: J04891540877, V94580952233, T84682689248, M22180503461
- `onCalcBaggagePrice()` is called as an `onKeyTyped` event for the weight text field to always show the current price
 - `onClear()` clears all fields and resets the radio button
 - `onAdd()` grabs all data, creates a passenger, saves it to the array, and shows it in the text area.
 - All exceptions must be caught and error messages shown to the user

Make sure that no errors occur and inform the user if there are errors he needs to take care of. You can find a demo video here.

GUI, Exceptions, LocalDate, Static

Demo Output:

Airport Registration!

First name: Nicole

Last name: Mayer

Birth date: 22.08.2004

☐ Passport ☒ ID card

ID card: M22180503461

Baggage weight:

Additional Price:

Loibner, Johannes - 31 years old: P123456; 60.00 €
Mayer, Nicole - 20 years old: M22180503461; 0.00 €

Exa_501_CarFactory_v2

Create the classes according to the class diagram. The aim of the program is to create and change car data.

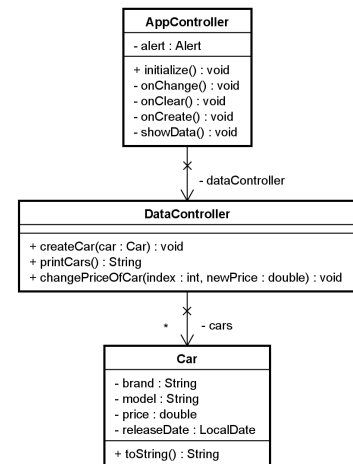
The Car class is a data class and contains a constructor and all other needed methods. The toString() method returns the object in the following format:

e.g.: Tesla-Model Y released in 02/2025 costs € 65000.00

Within the DataController class, there is a Car-List holding Car objects. The list is initialized within the constructor. The createCar() method adds a new car to the list and the changePriceOfCar() method changes the price of the car at the passed index. The printCars() method returns all cars in the list. Each car at a new line with the index at the beginning, starting with 1. You should use the toString() method of the Car class here.

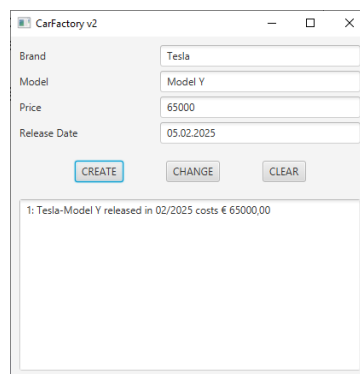
The ApplicationController holds all methods to handle clicks on the buttons. You are asked to register the action events in code and NOT in the FXML file. The showData() method displays all cars on the textarea.

Make sure that no errors occur and inform the user if there are errors he needs to take care of. You can find a demo video here.



LocalDate, List, in-code registration, Exception handling

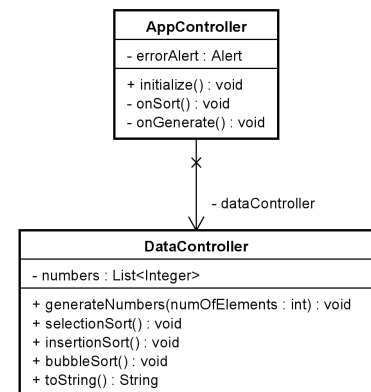
Demo Output:



Exa_502_NumberSorting_v2

Create a new project and all classes according to the class diagram. The aim of this program is to randomly create numbers and sort them by using different sorting algorithms.

The DataController is responsible for holding the numbers in a List of Integers. The 3 methods implement the according sorting algorithms as you learned them in LOAL. The toString() method returns a String containing all numbers, separated by a comma (,), and 10 elements per line. This is shown in the textarea. The generateNumbers() method is responsible to get rid of all elements in the array and create new elements. The number of elements is passed to the method. Make sure that the random numbers follow this interval: $0 < \text{random number} < \text{numOfElements}$.



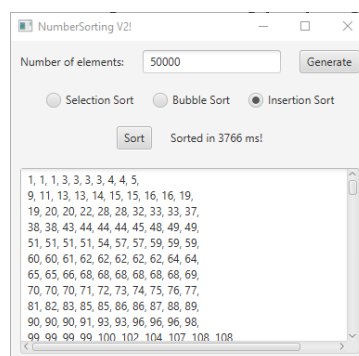
All action events in the AppController() need to be registered by hand and NOT within the FXML file. All instance variables are initialized in the initialize() method. All other method - button relations should be clear. Track the time for each sorting and write the result to the label as shown in the demo output / video.

Use this exercise to experiment a little bit with the performance of the algorithms. You can compare the needed time for unsorted and already sorted sets of data. A good starting point is with an amount of 50000+ elements.

Make sure that no errors occur and inform the user if there are errors he needs to take care of. You can find a demo video here.

List, in-code registration, Exception handling, Sorting algorithms, Time

Demo Output:

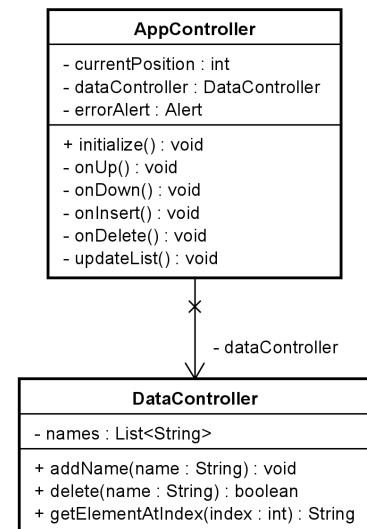


Exa_503_NameList

Create a new project and all classes according to the class diagram. Within this program, names are being managed. You can add and delete names, and also navigate through the list with some buttons.

The DataController holds the list (LinkedList) and implements the methods to add a name and to delete a name. A name can only be deleted, if it is present in the list, otherwise false is returned. Finally, there is a method to get the name a defined index getElementAtIndex.

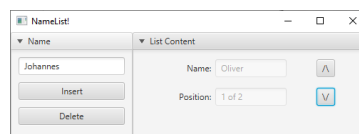
The ApplicationController initializes all instance variables in the initialize() method. All methods according to button clicks are created and the registration is done in code and NOT in the FXML file. The updateList() method is responsible to update what is shown the right two text fields. This should be called at the beginning and after every insert or delete.



Make sure that no errors occur and inform the user if there are errors he needs to take care of. You can find a demo video here.

List, in-code registration, Exception handling

Demo Output:

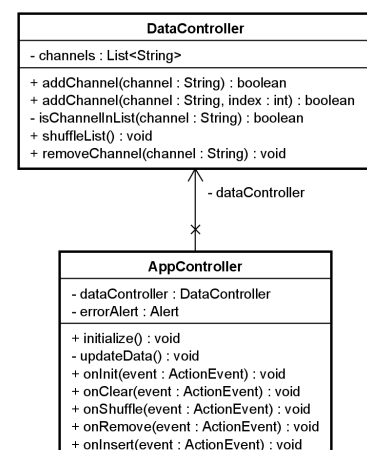


Exa_504_ChannelList

Create a new project and all classes according to the class diagram. This program manages tv channels in a List in combination with a ListView.

The DataController holds the List of channels (String). A new channel is only added, if it is not already in the list. Use isChannelInList() to check this. A new channel is either added at the end of the list, or at a certain defined index. If the insert was done, TRUE is returned, FALSE otherwise. A channel can also be removed by its name. The shuffleList() randomly shuffles all elements inside the list.

In the ApplicationController class, all instance variables are initialized within the initialize() method. The updateData() method is used to always update the data

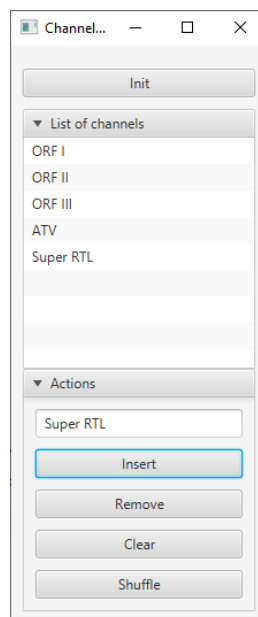


in the `ListView`. When the `Init` button is clicked, 4 channels are inserted into the list. You can choose which ones. By pressing the `Insert` button, a new channel is inserted into the list, if it isn't already there. If nothing is selected, the new channel is inserted at the end, otherwise, it is inserted at the position of the first element which is selected. With the help of the `Remove` button, all selected channels are deleted. `Clear` just removes all elements from the list and `Shuffle` randomly rearranges all the elements.

Make sure that no errors occur and inform the user if there are errors he needs to take care of. You can find a demo video [here](#).

List, `ListView`, `ObservableList`, Exception Handling

Demo Output:

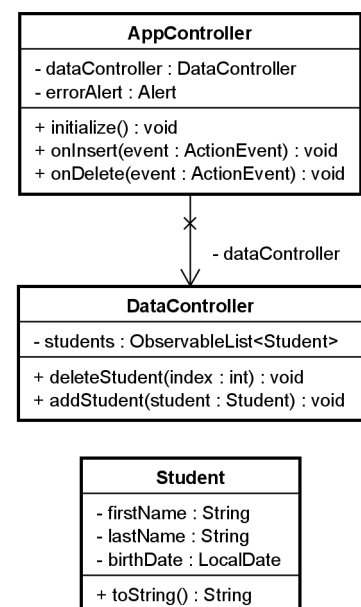


Exa_505_StudentList

Create a new project and all classes according to the class diagram. Students can be added and deleted and displayed within a list.

The class `pojos.Student` represents one Student. The `toString()` method returns the representation of a student within the list view. Implement to `Comparable` interface to prepare a sorting for students by first name and by last name.

The `DataController` holds an `Observable List` of students and implements the methods `addStudent()` and `deleteStudent()`. Make sure that list is always sorted after a new student is added. Create you own `Comparator` to sort the students by it's last name in a descending order. Additionally, create a third version of sorting the students by using an



Inner Anonymous Class to sort the students by it's first name.

Finally, the `AppController` implements all handler methods for button clicks. The instance variables need to be initialized within the `initialize()` method. Also, the connection of `List` and `ListView` needs to be done in here.

Make sure that no errors occur and inform the user if there are errors he needs to take care of. You can find a demo video [here](#).

List, ListView, ObservableList, Exception Handling, Comparator, Comparable

Demo Output:



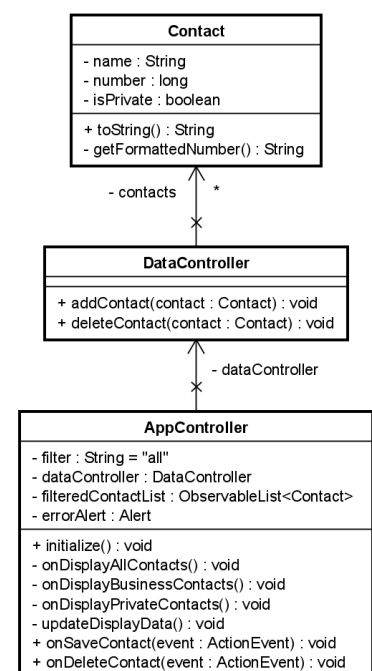
Exa_506_ContactManagement

Create a new project and all classes according to the class diagram. This program manages your contacts and helps to view business, private, or all contacts by filtering the list. It is possible to add and delete contacts.

The class `Contact` represents one contact and contains name, number and a value defining if it is a private or a business contact. The `getFormattedNumber()` method returns the number as shown in the demo output. This method needs to be used in the `toString()` to create the correct contact representation. The `Comparable` interface needs to be implemented. The list view needs to show all contacts always sorted by their name.

The `DataController` holds the list of contacts and implements methods for adding and deleting a contract.

All instance variables in `AppController` need to be initialized in the `initialize()` method. The `filtereContactList` needs to be connected to the list view and holds the currently shown data. At the beginning, the standard filter is set to `all`. Additionally, the context menu needs to be created.

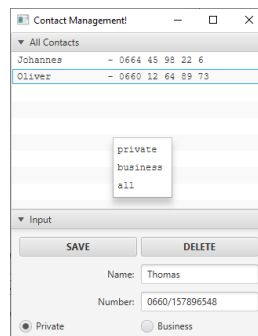


updateDisplayData() is used to update the observable list depending on the set filter. The onDisplay...() methods are executed according to the selected context menu item and set a value for the filter variable. Finally, onSaveContact() and onDeleteContact() are called on the button clicks.

Make sure that no errors occur and inform the user if there are errors he needs to take care of. You can find a demo video [here](#).

List, ListView, ObservableList, Exception Handling, Comparable, ContextMenu, in-code event registration, filter

Demo Output:



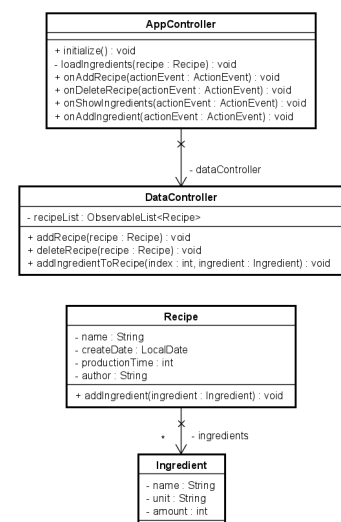
Exa_507_RecipeManagement

Create a new project and all classes according to the class diagram. The aim of this program is to manage recipes and their ingredients.

The class Ingredient holds one ingredient of a recipe. One ingredient should be always shown in the defined format. Make sure that the list of ingredients is by default sorted by the amount (ASC) if the unit is equal and otherwise by the unit (DESC). Use the Comparable interface to realize that. Additionally, an ingredient is uniquely defined by the name.

The Recipe class holds all information according the recipe and a list of ingredients. The ingredients list is empty at the beginning and can be filled using the GUI. This is what the addIngredient() method is used for. If an ingredient is already in the list, a DuplicateIngredientException needs to be thrown. Take a look at the GUI to see how the representation of an recipe in the list view looks like. A recipe is uniquely defined by the combination of name and author. The default sorting by the name (ASC) is defined using the Comparable interface.

The DataController holds an ObservableList of recipes. Make sure that the list is always sorted by the default order. The addRecipe() method adds a new recipe and throws a DuplicateRecipeException if the recipe is already in the list. deleteRecipe() just deletes the



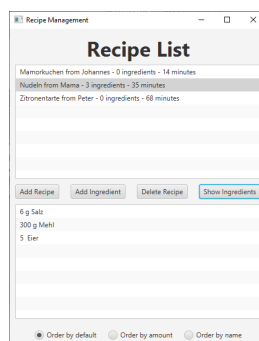
passed recipe from the list. The `addIngredientToRecipe()` method is used to add an ingredient to one of the recipes. The recipe is defined by its index.

Within the `AppController` the `initialize()` method is used to connect the recipes list with the `DataController`. The method `loadIngredients()` is used to show all the ingredients of the passed recipe in the defined list. Take care of the different ways of sorting, defined by the radio buttons. These additional sortings are realized using `Inner Anonymous Classes`. The `onAddRecipe()` method reacts on the click of the Add Recipe button and uses an `TextInputDialog` to ask the user for a new recipe in the following format: `name;author;productionTime`. Only if the format is correct and all data is fine, the new recipe can be added into the list. Make sure that the same recipe cannot be added twice. The `onDeleteRecipe()` method removes the selected recipe from the list. With the help of `onShowIngredients()` and `loadIngredients()` all ingredients of the selected recipe are shown in the list. The `onAddIngredient()` method asks the user for a new ingredient using an `TextInputDialog`. The user enters the ingredient in the following format: `amount-unit-name`. If all the data is correct, the ingredient is added. But only, if the same ingredient is not already in the list of the selected recipe. When adding an ingredient, the number of ingredients in the recipe list is updated, the view of the ingredients list is only updated, if the `showIngredients` button is pressed.

Make sure that no errors occur and inform the user if there are errors he needs to take care of. You can find a demo video [here](#).

List, ListView, ObservableList, Exception Handling, Comparable, Equals, Contains, TextInputDialog

Demo Output:



Exa_601_Decryption

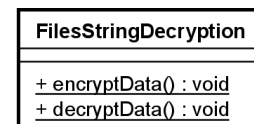
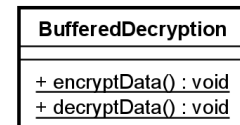
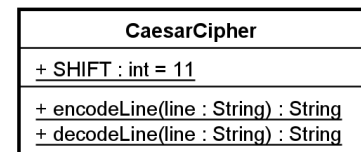
Create a new java console project and create all the classes and methods according to the class diagram. The aim of this program is to perform the Caesar Cipher with different ways of reading and writing.

In the src package create a package files and in there a package output. Download this file and copy it into the files folder. This is the file to always start with.

The class CaesarCipher implements two methods. encodeLine() uses the Caesar Cipher to encode the passed String and return it. The decodeLine() method takes an encrypted line and returns the process, so that the plain text of this line is returned. These methods must be used in all of the three tester classes.

The three classes ...Decryption all do the same thing. They read the file data.txt, encode the text and write the encoded file. Then they read the encoded file, decode it, and write the decoded file.

This means that each method in each of the classes reads one file and writes one file. The only difference is in how the reading and writing is implemented.



BufferedDecryption: Use BufferedReader, BufferedWriter, FileReader, FileWriter only in this class. The encrypt() method writes the bufferedEncrypted.txt file and the decrypt() method writes the bufferedDecrypted.txt file. Both are placed in the output folder.

FilesStringDecryption: Use Files only in this class. Make sure to read the whole file as one String. The encrypt() method writes the filesStringEncrypted.txt file and the decrypt() method writes the filesStringDecrypted.txt file. Both are placed in the output folder.

FilesListDecryption: Use Files only in this class. Make sure to read the whole file into a list. The encrypt() method writes the filesListEncrypted.txt file and the decrypt() method writes the filesListDecrypted.txt file. Both are placed in the output folder.

A sample of both files is shown in the demo output!

BufferedReader, BufferedWriter, FileReader, FileWriter, File, Files, Path, Static

Demo Output:

```
The Caesar cipher is one of the oldest known encryption techniques.
It was named after Julius Caesar, who used it to protect his military messages.
This cipher works by shifting each letter in the plaintext by a fixed number of positions
down the alphabet.
```

```
_sp+Nlp~1}+nt{sp}+t~+zyp+zq+!sp+zwop~!+vyzy+pyn}{!ltzy+!pnsyt|p~9
T!l+1~+ylxpo+lq!p}+Uwt~+Nlp~1}7+sz+~po+t!+!lz+{ }z!pn!+st~+xtwt!l}+xp~~!rp~9
_st~+nt{sp}+z}v~+m~+stq!ityr+plns+wp!lp}+ty+!sp+{wlty!p!+m+!+qtpo+yxmp}+zq+{z~t!ltzy~
ozy+!sp+lw{s!mp!9
```

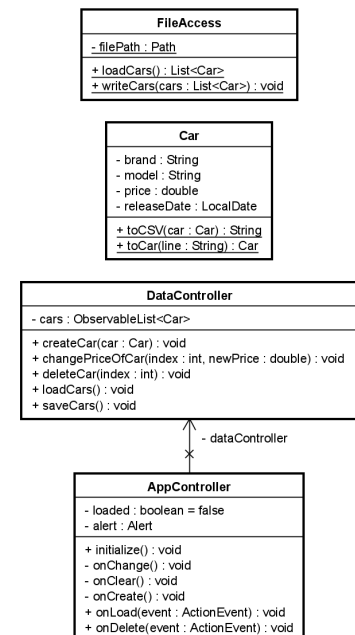
Exa_602_CarFactory_v3

This project should already be well known. With the new knowledge gained, this program needs to be refined. Please stick to the new class diagram and to the following refinements:

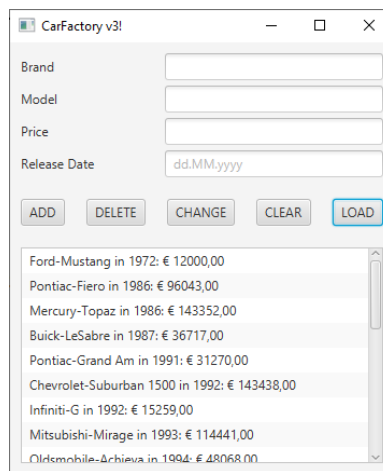
- **ListView:** The list in the DataController needs to be replaced by an ObservableList. Replace the TextArea by a ListView and make sure the new car representation (see the demo) is implemented.
- **Sorting:** All cars need to be sorted by the releaseYear at any time.
- **Duplicates:** A car is uniquely identified by the combination of brand and model. Make sure that the user receives the defined error message (see the demo) when trying to insert duplicate cars.
- **Delete:** Add a new button DELETE so that the user can delete one car that is selected in the list.
- **Load:** Add a new button LOAD so that the user has **once** the possibility to load cars from a carData.csv file. The functionality should be as follows:

- Download the carData.csv file from here and copy it to your resources folder into a new folder called files. This is the data file containing all the car data.
- At the beginning, the user should have to possibility to click the LOAD button to import all cars saved in the file. The new loadCars() method in the DataController should be used. To read data from the CSV file, implement the method loadCars() in the FileAccess class and the toCar() method in the Car class. The order of method calling is the following: ApplicationController.onLoad() -> DataController.loadCars() -> FileAccess.loadCars() -> Car.toCar(). All cars from the CSV file should be shown in the list view.
- The user can now go on with the program as before.
- Everytime the list of cars changes, the current state of data needs to be written back to the CSV file. This is done by calling the saveCars() method in the DataController. Here the FileAccess method writeCars() is used to write the file. Make sure to use the toCSV() method from the Car class to create **one line** for the CSV file.
- Use BufferedReader, BufferedWriter, FileReader, FileWriter for the implementation. When done, try to also implement the same functionality using the Files class.

Make sure that no errors occur and inform the user if there are errors he needs to take care of. You can find a demo video here.



Demo Output:



CarFactory v3!

Brand:

Model:

Price:

Release Date:

ADD DELETE CHANGE CLEAR LOAD

Ford-Mustang in 1972: € 12000,00
Pontiac-Fiero in 1986: € 96043,00
Mercury-Topaz in 1986: € 143352,00
Buick-LeSabre in 1987: € 36717,00
Pontiac-Grand Am in 1991: € 31270,00
Chevrolet-Suburban 1500 in 1992: € 143438,00
Infiniti-G in 1992: € 15259,00
Mitsubishi-Mirage in 1993: € 114441,00
Oldsmobile-Achieva in 1994: € 48068,00

Exa_603_MyShop Part 1

This project consists of two parts. Part one is just dealing with parts besides the GUI. Therefore you are asked to create a `main()` method in each class to test the methods in there.

The main goal of this program is to represent a shop. Products and categories are loaded from a csv file. A user can put items into the cart and when done, save the invoice for this cart as an HTML file. A sample HTML file can be downloaded here, the predefined CSS file (don't change that one) here, and the CSV file here. Place the CSV file into a new folder in the resources tree.

First, take a look at the HTML file. The different sections define the different parts of the program. Such a file should be created for every invoice export. The company name is always the same. The user name and the products may change.

The User class represents a user. The constructor takes all the attributes. The `toHTML()` method returns the user section in the following format:

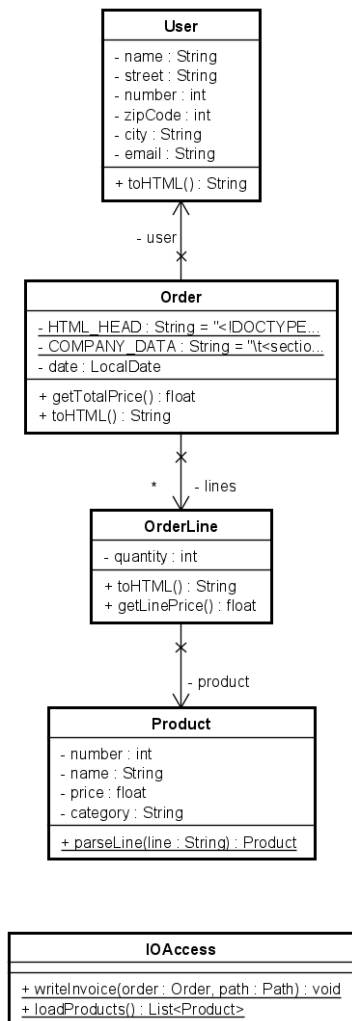
```
<section id="invoiceAddress">
  <h1>Johannes Loibner</h1>
  <p>Grazer Strasse 202</p>
  <p>8430 Leibnitz</p>
  <p>lr@htl-kaindorf.at</p>
</section>
```

Make sure to create a `main()` method and test the output of `toHTML()`.

The class Product represents a product which is uniquely defined by the number. Make sure to use the Comparable interface to sort them by name. The `parseLine()` is used to create new product object from one line in the csv file. You might also want to test this method.

The OrderLine class represents one row in the product table in the HTML file. It is uniquely identified by the product which means, that there cannot be two lines with the same product. In that case, the amount is increased. The `getLinePrice()` method calculates the price for that row defined by the product price and the amount. The `toHTML()` method returns one row of a product in the following format:

```
<tr>
  <td>3681</td>
  <td>Personal Blender with Travel Cup</td>
  <td>3.99</td>
  <td>3</td>
```



```

        <td>11.97</td>
    </tr>

```

Make sure to test this in a `main()` method before you go on.

Finally, the `Order` class combines everything. The two constants contain the HTML code for the header and the company data section as this data never changes. The constructor only holds the user and the date. The `toHTML()` method returns the whole content of the HTML file. Please make sure that the structure is exactly as defined in the HTML file. Create a `main()` method to test all the functionality.

In the `IOAccess` class, the `writeInvoice()` method is used to write a file. Test if this works for any order passed.

The `loadProducts()` method loads all the products from the CSV file using the `Files` class. A list of products is returned. Make sure to also test this method before you go on with the next exercise.

File, Files, Path, Static, List, ObservableList, Comparable, FileChooser

Exa_603_MyShop Part 2

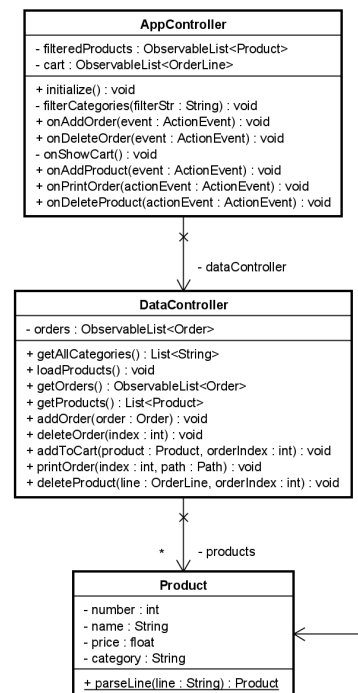
This is the second part of the program containing the controllers and the GUI.

The `DataController` holds a list of orders and a list of products. Use the correct list types here. The constructor initializes both of the instance variables. The `getAllCategories()` method returns a list of all categories present. Make sure that no categories are duplicated.

The `loadProducts()` method loads all products from the CSV file and adds them to the product list. Make sure that the list is always sorted. There are two methods to add and delete a product. They should be straight forward to implement.

`addToCart()` adds a new product to an order. Make sure to only create a new `OrderLine` if the product is not in the order yet. Otherwise, the amount for the `OrderLine` needs to be changed. The same principle is true for `deleteProduct()`. If a product reaches an amount of 0, the according `OrderLine` needs to be deleted!

With `printOrders()` method is responsible to create the HTML file at the specified location.



The `AppController` also holds lists for the filtered products and the cart. They need to be updated in methods. Within the `initialize()` method all instance variables are initialized and connected to the related list views. The products are loaded and displayed and the context menu for all categories is created. When filtering, the `filterCategories()` method is called and the content of the product list view changes. Don't forget to include the `all` option. As soon as an order is selected into the orders list view, the `onShowCart()` method is called and the list view with the cart is updated. To add a new order with `onAddOrder()` all user fields need to be correct. The current date is used

for the order to create. Deletion can be achieved by calling `onDeleteOrder()` from the button press.

With the two arrow buttons, products can be added to cart of the selected order or deleted. Use the two methods `onAddProduct()` and `onDeleteProduct()` for these two buttons.

Finally, when the PRINT button is pressed, `onPrintOrder()` is called. Use the `FileChooser` to ask the user for a location and a file name and save the invoice for the selected order.

Make sure that no errors occur and inform the user if there are errors he needs to take care of. You can find a demo video [here](#).

File, Files, Path, Static, List, ObservableList, Comparable, FileChooser

Demo Output:

The screenshot shows a Java Swing window titled "MyShop". It contains several sections:

- User Information:** A form with fields for Name (Thomas Mayer), Street (Grazer Straße), Number (202), Zip Code (8430), City (Leibnitz), and Email (lr@htl-kaindorf.at).
- Orders:** A list showing two orders: "Johannes Loibner: 05.04.2025" and "Thomas Mayer: 05.04.2025". Below the list are three buttons: "Add Order", "Delete Order", and "Print Order".
- Shopping Cart:** A list showing three items: "1 x Balsamic Vinaigrette: 2,69", "1 x Beef Enchilada Casserole: 8,99", and "2 x Beef Enchilada Casserole: 17,98".
- Products:** A list showing five products: "1708: Artisan Flatbreads - 2,99", "6375: Asian Noodle Salad Kit - 4,99", "2959: Balsamic Vinaigrette - 2,69", "8050: Beef Enchilada Casserole - 8,99", and "2288: Beef Enchilada Casserole - 8,99".

Below the Shopping Cart and Products sections are two arrow buttons: a blue button with an upward arrow (Add) and a grey button with a downward arrow (Delete).

Exa_701_VendingMachine

This program simulates a vending machine. No GUI is needed here - a simple console application should be created. Make sure to create all classes according to the class diagram. The constructors always initializes all the instance variables.

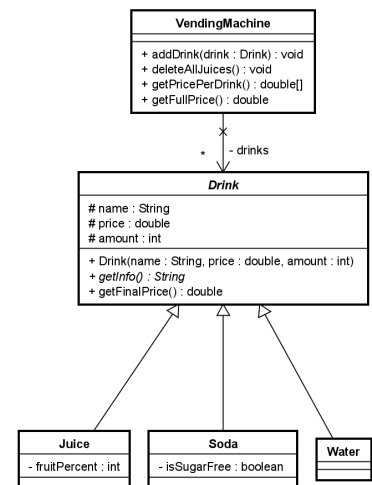
The `getInfo()` methods returns a `String` representing the object - the amount is **NOT** printed here!! The format can be seen in the demo output. Make sure that `getFinalPrice()` returns the final price of a drink, taken the price and the amount into account.

The class `VendingMachine` holds some methods (including the `main`). These methods are described here:

The method `addDrink()` adds a new drink to the list. `deleteAllJuices()` makes sure, that all `Juice` objects are deleted from the list. The method `getPricePerDrink()` returns an array with three numbers in there. Each number represents the total price of each of the subclasses. Finally, the `getFullPrice()` method returns the full price of all drinks in the list.

Within the `main` method you should test your program and produce the output in a format as shown below. Add any drinks you like to test your program. Make sure to also test and use the `deleteAllJuices()` method.

Make sure that no errors occur and inform the user if there are errors he needs to take care of.



Inheritance (extends, abstract, override), List

Demo Output:

```
---- Vending Machine ----
The following drinks are selected:
2 x Vöslauer (water) costs 1,25 €.
5 x Evian (water) costs 0,99 €.
3 x Spezi (soda) costs 1,88 €.
2 x Cola Zero (soda, sugarfree) costs 1,45 €.
1 x Organge (juice, 80%) costs 2,99 €.

The waters cost 7,45 €
The juices cost 2,99 €
The sodas cost 8,54 €
You have to pay: 18,98 €!
```

Exa_702_HomeDevices

The aim of this program is to show home devices in a list. By selecting items from the list, detailed information is shown and some parameters can be changed: turn on devices, turn off devices, set brightness, The CSV file which needs to be read, can be found here.

First, create all classes according to the class diagram and make sure to create the GUI. You might need Shapes to define the circles.

IO_Access: This class contains only one method which returns a list of all Appliances found in the CSV file. Use the `parseLine()` from the `Appliance` class to parse each of the lines in the file. Make sure to test if the reading process works as intended.

Lamp: The `setBrightness()` method sets the new brightness only, if the value is within the interval `[0;100]`. Depending on the brightness, also the amount of used power is calculated (50% brightness means half of the power).

Washer: The washer is very efficient. It only needs 80% of its defined power when used.

SmartControllable: This interface provides three methods. The `connect-` and `disconnect` methods just set the value of `connected` and the method `isConnected()` returns its current value.

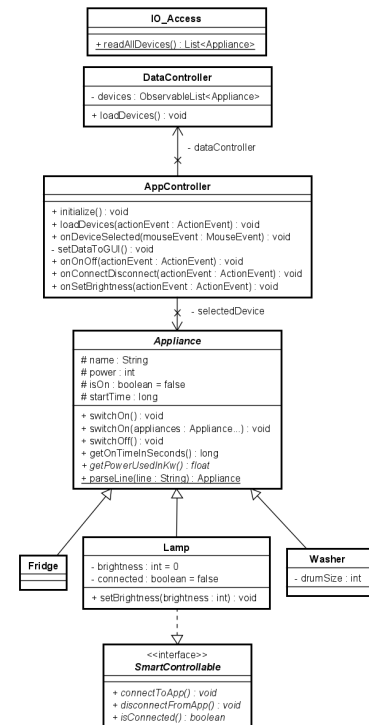
DataController: This class contains a list of all Appliances and a method `loadDevices()` to fill it with all elements from the file. Make sure to use the `Comparable` interface to sort the list ascending by the power of the Appliances.

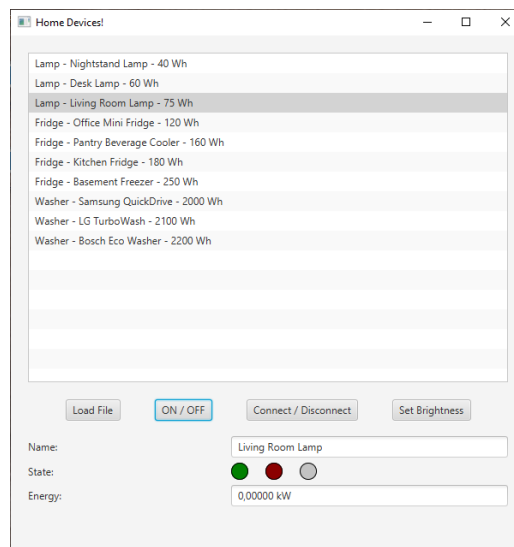
AppController: With the Load file button, all Appliances from the CSV file are loaded into the list. Make sure to stick to all the formatting. You might have recognized, that some buttons and fields only appear with certain types of Appliances selected. Make sure to also stick to these rules. The `setDataToGUI()` methods is responsible for setting all the data in the text fields and coloring everything. Using an event which fires when a mouse click is recognized in the list view might be useful.

The first colored circle is red if the selected device is OFF and green otherwise. The second circle is dark red if the devices is not connected and light green otherwise. The last circle indicates the brightness of lamps. If the brightness is 0, the circle is black. If the brightness is 100 the circle is white. In between, there are different shades of grey used.

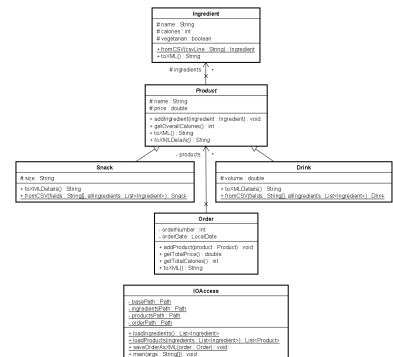
When creating the GUI, the following methods might be useful: `setVisible()`, `setManaged()`, `setFill()`, `Color.???`, `Color.rgb()`.

Make sure that no errors occur and inform the user if there are errors he needs to take care of. You can find a demo video here.



Demo Output:**Exa_703_FastFood**

The aim of this program is to create a fast food store. Each order has some products which consist of different ingredients. The ingredients can be found in this CSV file and the products in that CSV file. The idea is to create an order and save the order to an XML file. A demo file can be downloaded [here](#). Make sure to test all of the functionalities in main methods. The final test should be done in the main method of the IOAccess class.



Ingredient: The method fromCSV() creates an Ingredient from a line in the CSV file. toXML() generates the String representing one ingredient in the XML file.

Product: The toXML() creates the XML String for a product as shown in the XML file. The toXMLDetails() method is implemented in the derived classes and returns the one element for their special details (e.g. <size>Medium</size> for the Snack).

Snack, Drink: Both classes implement the toXMLDetails() method. The fromCSV() method creates an object of the defined class from one line in the CSV file. A list of all existing ingredients needs to be passed here too.

Order: This class implements the toXML() method to create the String to be written to the XML file. See the demo file to find out what this method adds to the already existing ones.

IOAccess: This class implements all the methods to read and write the files. You must have a main method in here to test the whole functionality of the program.

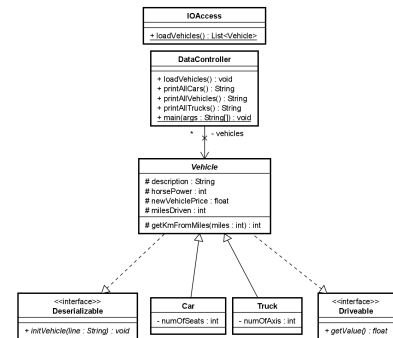
Optional: You may also create a JavaFX project and create a nice looking GUI for this project. Make sure that no errors occur and inform the user if there are errors he needs to take care of.

Exa_704_VehicleLeasing

You should create a very easy designed vehicle leasing. You should read the CSV file and print the results to the console in different ways.

pojos.Vehicle

This class represents a vehicle and should be created according to the class diagram. The method `getKmFromMiles()` converts miles to kilometers. The factor is $1\text{km} = 0.6214\text{mi}$. All vehicles are by default sorted by the type (CAR, TRUCK) and by the description (Description) if the type is the same.



pojos.Car, *pojos.Truck*

These classes represent specific vehicles. Be aware that $1\text{PS} = 1.36\text{KW}$.

pojos.Driveable

The method `getValue()` returns the current car value depending on the already driven kilometers. For the first 1000 Km for Trucks 30 %, for Cars 20 % of the current value is deducted. For the next 1000 Km 25 % or 19 % and so on. It ends if nothing more can be deducted. So a truck with a value of 1000 and 2000 Km driven, still has a value of 525, a Car would have a value of 648.

pojos.Deserializable

The method `initVehicle()` initializes a Truck or Car with the values from one CSV line.

controller.DataController

The controller hosts a list with all vehicles and a method to load them from the CSV file. The Main should be used to show all vehicles, all cars, and all trucks in the console (take a look at the demo)..

ALL:

CAR (with 5 seats): Audi A3 - 150 HP - 110,29 KW - 3651,19 (35000,00) EUR

CAR (with 4 seats): Audi A3 - 150 HP - 110,29 KW - 3651,19 (35000,00) EUR

...

CARS:

CAR (with 5 seats): Audi A3 - 150 HP - 110,29 KW - 3651,19 (35000,00) EUR

CAR (with 4 seats): Audi A3 - 150 HP - 110,29 KW - 3651,19 (35000,00) EUR

...

TRUCKS:

TRUCK (with 2 axis): Ford F-MAX - 500 HP - 367,65 KW - 25944,97 (85000,00) EUR

TRUCK (with 2 axis): Isuzu N-Series - 190 HP - 139,71 KW - 14651,28 (48000,00) EUR

...

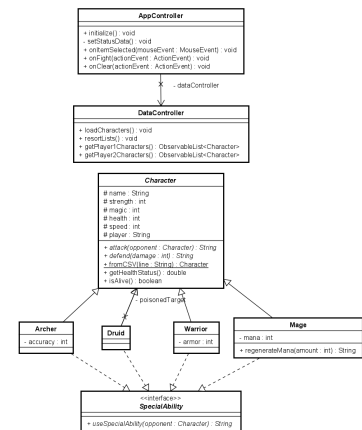
The CSV file can be downloaded [here](#).

Optional: You may also create a JavaFX project and create a nice looking GUI for this project. Make sure that no errors occur and inform the user if there are errors he needs to take care of.

Exa_705_DNDGame

You should create a DND like game for two players. The characters are read from a CSV file and are already assigned to the players. The user can view different properties and the health status of the characters. Finally, the characters can fight against each other.

Character: Create the class according to the class diagram. The constructor initializes all the instance variables. The `fromCSV()` method is used to convert one line from the CSV file to some character. The health status should be a value between 0.0 and 1.0 representing the health in percent. The representation of the Character in the lists is shown in the demo. Make sure to exactly stick to that. Finally, make sure that lists containing characters are always sorted by the health.



Archer: The archer's special ability is to focus and increase the accuracy by 10. Attack: In 25 % of the cases, the special ability is applied and prints `#name focuses: accuracy increases to #val`, otherwise the attack is done and the damage is half of the accuracy plus the strength. The output should be `#name shoots an arrow for #val damage..` Now the opponent has the ability to defend the attack. Defend: In 30 % of the attacks, nothing happens and `#name dodges the attack!` is printed. Otherwise the damage is deducted from the health and `#name is hit and loses #val HP.` is printed.

Druid: The druid's special ability is to poison the opponent. Attack: In 40 % of the cases, the special ability is applied and prints `#name poisons #opponentname!`, otherwise the attack is done and the damage is 1.3 times the magic plus the strength. The output should be `#name casts a nature spell for #val damage..` Additionally, the opponent suffers from a damage of 5, if the opponent was poisoned and `#name suffers 5 poison damage!` is printed. Now the opponent has the ability to defend the attack. Defend: The damage is deducted from the health and `#name takes #val damage.` is printed.

Mage: The mage's special ability is to heal itself by 30. Attack: In 30 % of the cases and if mana is over 19, the special ability is applied and prints `#name uses Heal and restores 30 HP. Mana left: #mana.` Otherwise, if mana is over 10, the attack is done and the damage is twice the magic and costs 10 mana. The output should be `#name casts a fireball for #val damage..` If there is not enough mana, `#name doesn't have enough mana.` is printed. Additionally, 5 mana is gained after every attack. Now, the opponent has the ability to defend the attack. Defend: The damage is deducted from the health and `#name takes #val damage.` is printed.

Warrior: The warrior's special ability is to block and increase armor by 5. Attack: In 20 % of the cases the special ability is applied and prints `#name uses Shield Block! Armor increases to #val..` Otherwise, the attack is done and the damage is 1.2 times the strength. The output should be `#name strikes with a sword for #value damage..` Now, the opponent has the ability to defend the attack. Defend: The damage is reduced by the armor value, deducted from health and

#name blocks with armor. Damage reduced to #val. is printed.

DataController: This class holds all the characters. They are read using the IOAccess class and within loadCharacters() they are additionally split up into the two lists.

AppController: This class should be self explaining. The setStatusData() method sets all the information for both of the selected characters. The progress bar can be set by using setProgress() with a value between 0.0 and 1.0. When 2 characters are selected, they can fight against each other. The character with the higher speed attacks the other one and vice versa. All the output is shown in the text area which can also be cleared. Make sure to also write DEAD in the list entry, if the character is not alive anymore. You can use listView.refresh() to reload the elements representation.

Make sure that no errors occur and inform the user if there are errors he needs to take care of. You can find a demo video here.

Inheritance (extends, abstract, override, interfaces), List, ObservableList, Comparable, Progressbar, GUI manipulation

Demo Output:

